

Project ID: 019 Prerequisite Group Project

Topic: Assessing the Effectiveness of Defensive Strategies at Each Stage of the Cyber Kill Chain in Simulated Cyberattacks

What to do: Practically simulate the 7 phases of the cyber kill chain, assess the effectiveness of the defense rate, response time, containment and impact, analyze and write your report.

Background

The Cyber Kill Chain framework, developed by Lockheed Martin (2011), outlines the seven stages of a cyberattack. This model helps organizations understand attacker tactics and apply appropriate defensive measures. This project simulates cyberattacks at each stage using Kali Linux, Burp Suite, and OWASP ZAP, targeting the OWASP Juice Shop, a deliberately vulnerable web application. The effectiveness of defensive strategies at each stage is evaluated.

Methodology

The penetration test employed a red-team/blue-team approach. The red team simulated attacks using tools like Burp Suite and OWASP ZAP, while the blue team monitored and defended the application. Each phase of the cyber kill chain was targeted with specific techniques:

- 1. Reconnaissance:** Both teams gathered information about the Juice Shop application and its environment. Both passive (OSINT) and active methods (OWASP ZAP), were used to gather information.
- 2. Weaponization:** The team identified and prepared tools to exploit discovered vulnerabilities.
- 3. Delivery:** Attack vectors were employed to deliver the exploit to the target application. This includes the use of Burp Suite to brute-force and obtain login credentials using a list of the most common passwords.
- 4. Exploitation:** Vulnerabilities such as broken authentication, broken access control, Sensitive Data Exposure and injection were leveraged to gain unauthorized access or control.
- 5. Installation:** Not applicable for this simulated test.
- 6. Command and Control (C&C):** Not applicable for this simulated test.

7. **Actions on Objectives:** The attackers achieved their objectives, such as data exfiltration or manipulation of the application using gained administrative access.

Disclaimer: This penetration test was conducted on a simulated environment and may not reflect the security posture of a real-world application.

Installation and Setup

Prerequisites: Node.js (LTS version recommended), npm (comes with Node.js), Git, Docker (optional, for containerized setup)

Set up Virtual Environment

- Install VirtualBox on your host machine.
- Download and install Kali Linux ISO and configure it in VirtualBox.
- Allocate sufficient resources (4GB RAM, 2 CPUs recommended).
- Install Burp Suite: `sudo apt install burpsuite`
- Update Kali Linux and install other tools if not pre-installed: `sudo apt update && sudo apt install nmap nikto gobuster`

Install OWASP Juice Shop

Method 1: Running with Node.js

- Install Node.js and npm on your host machine or another VM.
- Clone the Juice Shop repository: `git clone https://github.com/juice-shop/juice-shop.git cd juice-shop npm install npm start`
- Access Juice Shop at `http://localhost:3000` from Kali Linux.

Method 2: Running with Docker

- Pull the official Docker image: `docker pull bkimminich/juice-shop`
- Run the container: `docker run -d -p 3000:3000 bkimminich/juice-shop`
- Open a browser and navigate to `http://localhost:3000`.

Stage 1: Reconnaissance (Information Gathering)

The attacker searches through the About page and DevTools (main.js) to locate hidden functionalities. Real-world equivalent: Attackers perform open-source intelligence (OSINT), scanning public repositories, exposed endpoints, or metadata leaks.

Find the Hidden 'Score Board' Page

1. Open the browser Developer Tools and go to the Sources tab.
2. Locate and open the main.js file.
3. Use the search function to find occurrences of score, identifying the route mapping section.
4. Navigate to <http://localhost:3000/#/score-board> to reveal the hidden scoreboard.

Read Our Privacy Policy

1. Log in to the application with any user account.
2. Open the dropdown menu on your profile picture and choose Privacy & Security.
3. Navigate to <http://localhost:3000/#/privacy-security/privacy-policy>.

Stage 2&3: Weaponization (Preparing the Attack) & Delivery (Deploying the Exploit)

Here, The attacker scans for exposed directories using brute-force techniques (Reconnaissance). They identify and access sensitive logs (Weaponization). The log file is retrieved, providing potential sensitive information such as IPs, authentication logs, or even credentials (Delivery).

Gain access to any access log file of the server

1. Visit <http://localhost:3000/ftp> and notice the file [incident-support.kdbx](#) (which is needed for [Log in with the support team's original user credentials]) and indicates that some support team is performing its duties from the public Internet and possibly with VPN access.
2. Guess luckily or run a brute force attack with e.g. [OWASP ZAPs DirBuster plugin] for a possibly exposed directory containing the log files.
3. Following the hint to drill down deeper than one level, you will at some point end up with <http://localhost:3000/support/logs>.

4. Inside you will find at least one `access.log` of the current day. Open or download it to solve this challenge.

Stage 4: Exploitation (Executing the Attack)

The attacker uses weak or default credentials to exploit poor security hygiene. Real-world equivalent: Brute-force attacks, credential stuffing, or guessing common passwords.

View another user's shopping basket

1. Log in as any user.
2. Put some products into your shopping basket.
3. Inspect the *Session Storage* in your browser's developer tools to find a numeric `bid` value.
4. Change the `bid`, e.g. by adding or subtracting 1 from its value.
5. Visit `http://localhost:3000/#/basket` to solve the challenge. If the challenge is not immediately solved, you might have to F5-reload to relay the `bid` change to the Angular client.

Log in with the administrator's user credentials

1. Visit `http://localhost:3000/#/login`.
2. Use the following credentials:
 - Email: `admin@juice-sh.op`
 - Password: `admin123`
3. This exploits weak password security, logging into the admin account.

Admin stolen credentials

SQL Injection to Log in as Admin

1. Visit `http://localhost:3000/#/login`.
2. Use SQL injection payloads:
 - Email: `' OR 1=1--`
 - Password: Any value
 - OR Email: `admin@juice-sh.op'--`

3. This allows authentication as the administrator without a valid password. The attacker exploits SQL Injection to bypass authentication (‘ OR 1=1 —). Real-world equivalent: Exploiting insecure login forms using SQL Injection techniques.

Stage 5: Installation (Establishing Persistence)

The attacker locates the admin panel and logs in using previously stolen credentials. This could be part of installation if the attacker plants a backdoor or escalates privileges. Real-world equivalent: Attackers gaining access to a C2 (Command & Control) panel or deploying web shells.

Access the administration section of the store

1. Open the main.js in your browser’s developer tools and search for “admin”.
2. One of the matches will be a route mapping to path: “administration”.
3. Navigating to <http://localhost:3000/#/administration> will give a 403 Forbidden error.
4. Log in to an administrator’s account by solving the challenge.

Perform a persisted XSS attack without using the frontend application at all

Here, The attacker injects malicious JavaScript into the product description via direct API manipulation (Exploitation). The payload is stored in the database and executed when a user visits the affected page (Installation).

1. Log in to the application with any user.
2. Copy your **Authorization** header from any HTTP request submitted via browser.
3. Submit a POST request to <http://localhost:3000/api/Products> with
 - {“name”: “XSS”, “description”: “<iframe src=\\”javascript:alert(xss)\\”>”, “price”: 47.11} as body,
 - **application/json** as **Content-Type**
 - and **Bearer ?** as **Authorization** header, replacing the ? with the token you copied from the browser.
4. Visit <http://localhost:3000/#/search>.
5. An alert box with the text “xss” should appear.

6. Close this box. Notice the product row which has a frame border in the description in the *All Products* table.
7. Click the “eye”-button next to that row.
8. Another alert box with the text “xss” should appear. After closing it the actual details dialog pops up showing the same frame border.

Stage 6: Command & Control (C2)

No direct example from this simulation. However, an attacker could escalate the attack by uploading a malicious script in a real-world scenario.

Stage 7: Actions on Objectives (Achieving the Attack Goal)

Access the Administration Section

1. Open Developer Tools and search for “admin” in main.js.
2. Find a route mapping to path: "administration".
3. Navigate to <http://localhost:3000/#/administration>.
4. If forbidden, log in as an admin using the previously exploited credentials.

Access a Confidential Document

1. Click on Check out our boring terms of use from the About Us page (<http://localhost:3000/ftp/legal.adoc>).
2. Modify the URL to browse the directory: <http://localhost:3000/ftp>.
3. Open <http://localhost:3000/ftp/acquisitions.adoc> to solve the challenge. The attacker successfully bypasses directory restrictions (<http://localhost:3000/ftp/acquisitions.adoc>).
Real-world equivalent: Data breaches, exfiltration of sensitive files.

Perform an unwanted information disclosure by accessing data cross-domain

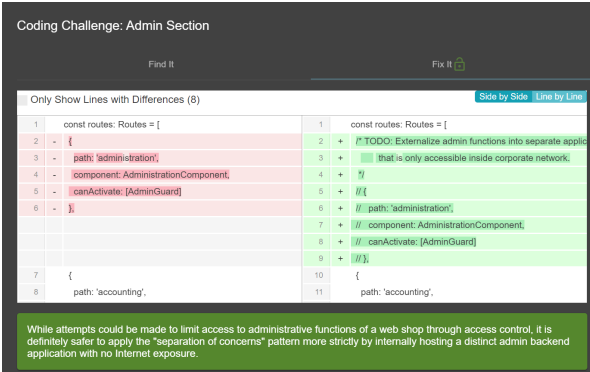
Here, the attacker exploits a misconfigured API that allows unauthenticated requests with JSONP (Exploitation). This vulnerability could lead to Cross-Origin Data Theft, where sensitive user information is accessed (Actions on Objectives).

1. Find a request to the `/rest/user/whoami` API endpoint. Notice that you can remove the “Authorization” header and it still works.
2. Add a URL parameter called “callback”. This will cause the API to return the content as a JavaScript fragment (JSONP) rather than just a standard JSON object.

Results

The effectiveness of the implemented defensive strategies was evaluated based on:

- **Defense Rate:** The percentage of attacks successfully blocked.
- **Response Time:** The time taken to detect and respond to an attack.
- **Containment:** The effectiveness in isolating and limiting the impact of a successful attack.



- **Impact:** The severity of damage caused by a successful attack was analyzed based on the (CIA) Triad security model rabilities were fully exploited.

Note: Since this was a simulated test, response time and containment measures weren't directly assessed.

Vulnerability	Red Team	Blue Team (Recommendations)	Defense Strategy
Sensitive Data Exposure	Accessed confidential documents via insecure FTP server	Eliminate the <i>/ftp</i> folder entirely. Move valid content and secure order confirmation PDFs.	<i>Defense Rate: 0%, Response Time: N/A, Containment: N/A, Impact: Confidentiality</i>
Broken Access Control	Gained access to the administration section through both URL guessing and (main.js) code analysis.	Implement role-based access control (RBAC) to restrict unauthorized access to administrative functionalities or separate the admin backend application from the public web application.	<i>Defense Rate: 0%, Response Time: N/A, Containment: N/A, Impact: Confidentiality, and Integrity</i>
Broken Authentication	Compromised administrator accounts by leveraging a password list and brute-force techniques within Burp Suite.	Enforce strong password policies aligned with NIST-800-63B recommendations. This includes a minimum password length of eight characters and a ban on common dictionary words.	<i>Defense Rate: 0%, Response Time: N/A, Containment: N/A, Impact: Confidentiality, Integrity, and Availability</i>
Injection	Guessed administrator and user accounts (e.g., "Jim") using SQL Injection	Implement prepared statements or stored procedures to prevent SQL injection attacks. <i>Using the built-in binding (or replacement) mechanism of Sequelize is equivalent to creating a Prepared Statement. This prevents tampering with the query syntax through malicious user input as it is "set in stone" before the criteria parameter is inserted.</i>	<i>Defense Rate: 0%, Response Time: N/A, Containment: N/A, Impact: Confidentiality, Integrity, and Availability</i>

Recommendations:

- Implement multi-factor authentication (MFA) to further strengthen login security.
- Regularly update the Juice Shop application to address known vulnerabilities.
- Conduct periodic penetration testing with a wider scope to identify additional vulnerabilities.
- Deploy an IDS/SIEM system to detect and respond to potential intrusions in real-time.

Conclusion

Our assessment highlights the importance of robust defenses at each stage of the Cyber Kill Chain. The OWASP Juice Shop demonstrated some basic security measures that prevented critical attacks. However, vulnerabilities were identified that could be exploited by a determined attacker to gain limited access to internal resources. Implementing the recommendations can significantly improve the Juice Shop's security posture.

Reference

Appendices

Sensitive Data Exposure (Code Fix)

Coding Challenge: Confidential Document

Find It Fix It

Only Show Lines with Differences (5) Side by Side Line by Line

```
1 - /* /ftp directory browsing and file download */
2 - app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true }));
3 - app.use('/ftp?!/quarantine/file', fileServer());
4 - app.use('/ftp/quarantine/file', quarantineServer());
5 -
6 1 /* /encryptionkeys directory browsing */
7 2 app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', { icons: true, view: 'details' }));
8 3 app.use('/encryptionkeys/file', keyServer());
9 4
10 5 /* /logs directory browsing */
11 6 app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view: 'details' }));
12 7 app.use('/support/logs/file', logFileServer());
13 8
14 9 /* Swagger documentation for B2B v2 endpoints */
15 10 app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
16 11
17 12 app.use(express.static(path.resolve('frontend/dist/frontend')))
```

Getting rid of the /ftp folder entirely is the only way to plumb this data leakage for good. Valid static content in it needs to be moved to a more suitable location and order confirmation PDFs had no business to be placed there publicly accessible in the first place. Everything else in that folder was just accidentally put & forgotten there anyway.

Correct Fix Fix 2 Close Submit

Broken Authentication & Injection (Code Fix)

Coding Challenge: Login Admin

```
1 + import { BasketModel } from ".././././models/basket";
2 +
3
4 1 module.exports = function login () {
5 2   function afterLogin (user: { data: User, bid: number }, res: Response, next: NextFunction) {
6 3     BasketModel.findOrCreate({ where: { UserId: user.data.id } })
7 4     .then([basket]: [BasketModel, boolean]) => {
8 5       const token = security.authorize(user)
9 6       user.bid = basket.id // keep track of original basket
10 7       security.authenticatedUsers.put(token, user)
11 8       res.json({ authentication: { token, bid: basket.id, umail: user.data.email } })
12 9     }).catch((error: Error) => {
13 10       next(error)
14 11     })
15 12   }
16 13 }
17 14
18 15
19 16 return (req: Request, res: Response, next: NextFunction) => {
20 17   models.sequelize.query(" SELECT * FROM Users WHERE email = '${req.body.email} ' AND password = '${
21 18   models.sequelize.query(" SELECT * FROM Users WHERE email = $1 AND password = $2 AND deletedAt IS N
22 19   { bind: [ req.body.email, security.hash(req.body.password) ], model: models.User, plain: true })
23 20   .then([authenticatedUser: { data: User }]) => {
24 21     const user = utils.mapResultToUser(authenticatedUser)
```

Using the built-in binding (or replacement) mechanism of Sequelize is equivalent to creating a Prepared Statement. This prevents tampering with the query syntax through malicious user input as it is "set in stone" before the criteria parameter is inserted.

Correct Fix Fix 3 Close Submit

Account 0 Your Basket 0 EN

admin@juice-sh.op

Orders & Payment

Privacy Policy

Request Data Export

Request Data Erasure

Change Password

+2 2FA Configuration

Last Login IP

Privacy & Security

Logout

est Juice Shop salesman Artwork 5000€

Add to Basket

Coding Challenge: Login Jim

```
1 module exports = function login () {
2   function afterLogin (user: { data: User, bid: number }, res: Response) {
3     BasketModel.findOrCreate({ where: { UserId: user.data.id },
4       then(([,basket], [BasketModel, boolean]) => {
5         const token = security.authorize(user)
6         user.bid = basket.id // keep track of original basket
7         security.authenticatedUsers.put(token, user)
8         res.json({ authentication: { token, bid: basket.id, email: user.data.email } })
9       }) catch((error: Error) => {
10        next(error)
11      })
12    }
13  }
14  return (req: Request, res: Response, next: NextFunction) => {
15    models.sequelize.query('SELECT * FROM Users WHERE email = ? AND password = ?')
16    .then(([,user], [UserModel, boolean]) => {
17      if (boolean) {
18        const token = security.authorize(user)
19        user.bid = basket.id // keep track of original basket
20        security.authenticatedUsers.put(token, user)
21        res.json({ authentication: { token, bid: basket.id, email: user.data.email } })
22      } else {
23        res.status(401).json({ error: 'Invalid email or password' })
24      }
25    }) catch((error: Error) => {
26      next(error)
27    })
28  }
29 }
```

Using the built-in binding (or replacement) mechanism of Sequelize is equivalent to creating a Prepared Statement. This prevents tampering with the query syntax through malicious user input as it is "set in stone" before the criteria parameter is inserted.

Correct Fix
Fix 3

👍 🗑️ ✖ Close Submit ✓

Broken Access Control (Code Fix)

Administration	
Registered Users	Customer Feedback
 admin@juice-sh.op	1 I love this shop! Best products I have! Highly recommended! (***@juice-sh.op) ★★★★★
 jim@juice-sh.op	2 Great shop! Awesome service! (***@juice-sh.op) ★★★★★
 bened@juice-sh.op	3 Nothing useful available here! (***@juice-sh.op) ★
 sara.kenneth@gmail.com	21 Please send me the juicy chocolate BT in my wallet at july 1st - purpose: betray marriage... ★
 alex@juice-sh.op	Incompetent customer support! Can't even upload photo of broken purchase... ★★
 support@juice-sh.op	This is the store for awesome stuff of all kinds! (anonymous) ★★★★★
 mario@juice-sh.op	Never gonna buy anywhere else from now on! Thanks for the great service! (anonymous) ★★★★★
 mc_silverwind@juice-sh.op	Keep up the good work! (anonymous) ★★★
 j227142@juice-sh.op	
 wuwind@juice-sh.op	